DATABASE ARCHITECTURE STUDY

Discord Database Architecture

How Cassandra, PostgreSQL, Redis, Elasticsearch, Kafka, and S3 work together to move billions of messages and millions of voices in real time.

Prepared By

Sree Ram R G — 24104026
Bala Gurunathan H — 24104008

The Challenge of "Discord Scale"

Why Polyglot Persistence?

Discord cannot rely on a single database because different parts of the platform require different storage solutions.

Some services need ultra-fast message retrieval, others need strong consistency for accounts, while analytics processes billions of events.

So Discord follows a Polyglot Persistence approach — picking the best-fit database for each workload.

Main Goals

- High Availability — Communication continues even during failures.
- Scalability — Support millions of concurrent users.
- Low Latency — Messages and voice feel instant.
- Consistency — User accounts and permissions stay accurate.
- Flexibility — Each microservice uses the database suited to its workload.

These goals often pull in opposite directions — for example, strong consistency usually costs some availability during a network partition — so the choice of database per service is really a choice of which trade-off to accept.

The Request Flow

1. Client

User opens Discord on Desktop, Mobile, or Browser.

2. API Gateway

Request reaches the gateway for authentication and routing.

3. Microservices

Routed to Authentication, Messaging, Voice, Friends, Notifications.

4. Databases

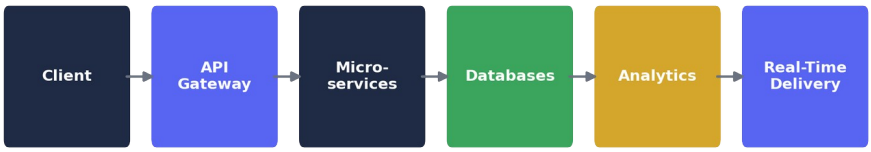
Services read/write PostgreSQL, Cassandra, Redis, Elasticsearch.

5. Analytics

User activity streams into analytics and monitoring pipelines.

6. Real-Time Delivery

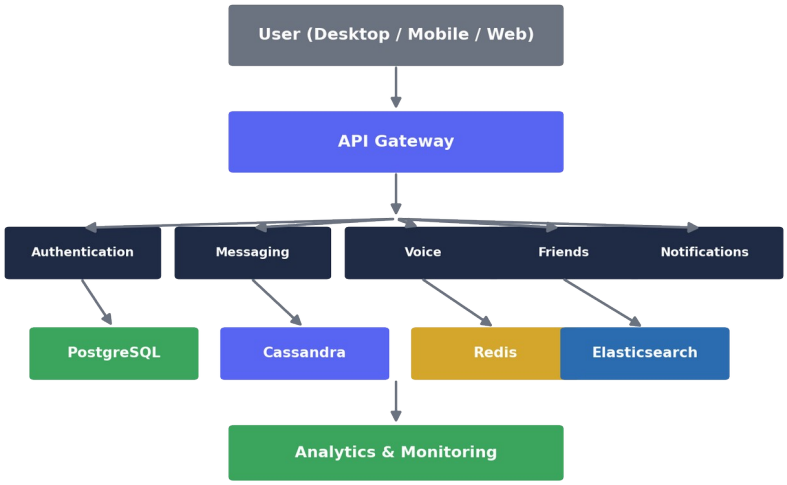
Messages and voice are delivered live via gateway servers.



Each step hands off to the next in milliseconds — the gateway never waits on analytics, so a slow reporting job can never delay a message being delivered.

Architecture Diagram

The diagram below shows how a single request fans out: one client call passes through the gateway, gets routed to the right microservice, and each microservice talks to the database best suited to it.



The Workhorse — Wide-Column Database

Cassandra

Handling billions of messages across a globally distributed cluster.

Used For	Why Cassandra?
• Chat messages • Message history • Activity logs • Presence information • Large-scale event storage	• Horizontal scalability • High write throughput • Distributed storage • High availability • Fast retrieval of conversation history

Discord originally migrated its messaging backend to Cassandra because relational databases struggled with message volume.

The Banker — Relational Database

PostgreSQL

Keeping accounts, permissions, and billing data accurate and consistent.

Used For	Why PostgreSQL?
• User accounts • Authentication • Friend lists • Server metadata • Roles & permissions • Billing information	• ACID transactions • Strong consistency • Reliable relational storage • Excellent indexing • Supports complex SQL queries • Ensures data integrity

PostgreSQL is the system of record wherever correctness matters more than raw throughput.

The Accelerator — Key-Value Cache

Redis

Shaving milliseconds off every request with in-memory speed.

Used For	Why Redis?
• Session cache • Online presence • Frequently accessed user data • Temporary auth tokens • Rate limiting	• Extremely low latency • In-memory storage • Reduces database load • Improves response time • Ideal for fast-changing data

Redis sits in front of the slower stores, absorbing traffic that would otherwise hit Cassandra or PostgreSQL directly.

Database Choice Rationale

Module	Database	Type	Reason
Chat Messages	Cassandra	NoSQL	High write scalability
User Accounts	PostgreSQL	SQL	Strong consistency
Presence Status	Redis	Key-Value	Ultra-fast access
Media Files	AWS S3	Object	Durable file storage
Search & Discovery	Elasticsearch	Search Index	Full-text search at scale

CAP Theorem Implications

Discord chooses databases based on service requirements — instead of forcing one CAP model onto every workload.

AP Model — Cassandra

- High availability
- Message storage
- Event logging

CP Model — PostgreSQL

- User management
- Authentication
- Billing

Speed-First — Redis

- Prioritizes speed
- Cached responses
- Temporary session storage

NoSQL Modeling: Cassandra

Discord stores messages according to access patterns instead of normalized relational tables.

Channel_ID → Message_ID → Sender_ID · Timestamp · Content · Attachments

Advantages

- Fast message retrieval
- No expensive JOIN operations
- High write performance
- Easily scalable across data centers

Key Idea: Discord denormalizes data because storage is cheaper than latency.

SQL vs NoSQL Comparison

Feature	PostgreSQL	Cassandra
Data Model	Relational	Wide Column
Transactions	ACID	Eventual Consistency
Scalability	Vertical + Limited Horizontal	Horizontal
Best For	User Accounts	Chat Messages
Query Language	SQL	CQL

Supporting Services

AWS S3 — Object Storage

- User-uploaded images & videos
- Attachments
- Server icons & emojis
- Backup storage

Durable, highly available storage for everything that isn't structured data. None of this content needs a database's query features — it just needs to be stored cheaply and served reliably, which is exactly what object storage is built for.

Elasticsearch — Search Engine

- Message search
- User search
- Server search
- Fast content discovery

Full-text search across billions of messages, in milliseconds. Cassandra is great at fetching messages by channel and time, but terrible at 'find every message containing this word' — that's the gap Elasticsearch fills.

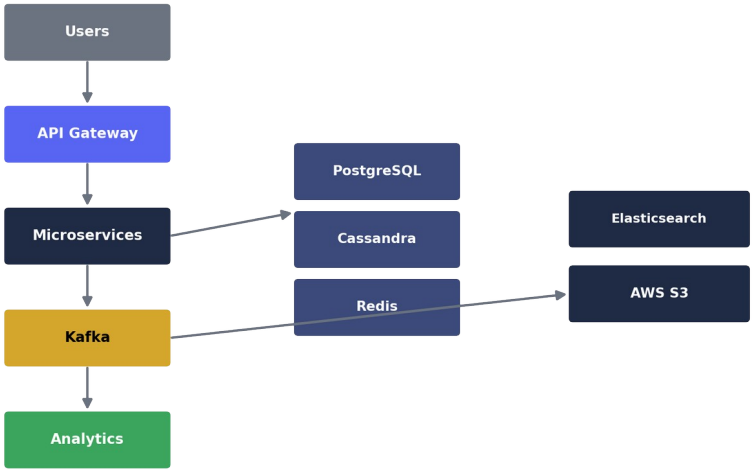
Kafka — Event Streaming

- Event streaming
- Message processing
- Monitoring
- Analytics pipeline

Moves millions of events between Discord's services in real time, so a spike in one service (say, a viral server) doesn't overload another service directly — Kafka buffers the load in between.

The Data Connectivity

Kafka is the glue in this picture: instead of every microservice calling every database directly, events flow through Kafka first, which keeps Cassandra, Redis, Elasticsearch, and S3 loosely coupled from the core request path.



Architecture Investigation Complete

Discord uses a polyglot database architecture because one database cannot efficiently handle every application requirement.

- Cassandra manages billions of chat messages.
- PostgreSQL stores user accounts and relational data.
- Redis accelerates frequently accessed information.
- Elasticsearch powers message search.
- AWS S3 stores uploaded media files.
- Kafka connects operational systems with analytics.

Final Conclusion: Discord achieves fast, scalable, reliable communication by choosing the right database for each workload — enabling real-time messaging, voice, and high availability for millions of users worldwide.